Operadores y Estructuras de Decisión en Java con IntelliJ IDEA

Taller Práctico Intensivo:

1. Guía de Inicio y Configuración del Entorno en IntelliJ IDEA

El dominio de la lógica de control constituye el verdadero motor del desarrollo de software en Java. Un profesional de la programación no solo escribe código ejecutable, sino que diseña estructuras capaces de tomar decisiones dinámicas, evaluar condiciones bajo algoritmos eficientes y garantizar el flujo correcto de los datos en tiempo de ejecución. Para sentar las bases sólidas de este taller práctico, configuraremos desde cero un entorno de desarrollo profesional en IntelliJ IDEA.

Instrucciones Paso a Paso para la Creación del Proyecto:

1. **Apertura del Entorno:** Inicie IntelliJ IDEA y seleccione la opción **New Project** (Nuevo Proyecto) en la pantalla de bienvenida o a través de **File** > **New** > **Project**.
2. **Configuración de Parámetros Generales:**
   - **Name (Nombre):** Ingrese exactamente `TallerUnidad1_2`.
   - **Location (Ubicación):** Elija un directorio de trabajo local con permisos de escritura.
   - **Language (Lenguaje):** Seleccione **Java**.
   - **Build System (Sistema de Construcción):** Seleccione **IntelliJ**.
   - **JDK (Java Development Kit):** Verifique que esté seleccionada una versión LTS activa de Java (JDK 17 o superior recomendada).
3. **Estructuración del Paquete Principal:**
   - En la ventana del proyecto (*Project Tool Window*), expanda la raíz y localice la carpeta `src`.
   - Haga clic derecho sobre `src`, seleccione **New** > **Package** y escriba el nombre estructurado del paquete: `com.taller.practico`.
4. **Creación de la Clase de Entrada:**
   - Haga clic derecho sobre el paquete recien creado (`com.taller.practico`), seleccione **New** > **Java Class** y defina la clase con el nombre `Main`.

Tips de Productividad en IntelliJ IDEA

- **Atajo `**psvm**` / `**main**`:** Dentro del cuerpo de la clase `Main`, escriba `psvm` (o `main`) y presione `Tab` o `Enter`. El IDE generará instantáneamente el bloque completo de la firma del método principal: `public static void main(String[] args)`.
- **Atajo `**sout**`:** Escriba `sout` y presione `Tab` dentro de cualquier método para generar automáticamente la sentencia de impresión estándar `System.out.println();`.

- **Compilación y Ejecución Rápida:** Presione la combinación de teclas `Shift + F10` en Windows/Linux (o `Control + R` en macOS) para compilar y ejecutar el archivo actual sin recurrir al ratón.

Una vez configurado nuestro entorno de desarrollo e inicializada la clase principal, estamos listos para pasar de la infraestructura básica a la evaluación precisa de expresiones matemáticas y lógicas.

2. Sección 1: Calentamiento de Operadores

En esta primera fase analítica, evaluaremos la precedencia de operadores, el comportamiento del operador residuo (`%`), las diferencias fundamentales entre el incremento prefijo (`++x`) y posfijo (`x++`), la promoción implícita de tipos de datos en operaciones aritméticas y la evaluación con cortocircuito (*short-circuit*) en operadores lógicos (`&&` , `||` , `!`).

Nota Pedagógica de Estado: Para la evaluación de la siguiente tabla, asuma de manera independiente para cada fila que las variables parten del estado inicial: `int x = 7;` y `boolean flag = false;` . Complete la columna de predicción mediante análisis mental o en papel antes de trasladar el código a su clase `Main` en IntelliJ IDEA.

Tabla de Evaluación de Expresiones

Expresión en Java	Predicción del Estudiante (Sin IDE)	Resultado Real en IntelliJ IDEA
`int seg = 86400 % 3600;`	0	0
`int y = ++x + x++;`	9	16
`double div1 = 15 / 2; double div2 = 15.0 / 2;`	7.5; 7.50	7; 7.5
`boolean res = flag && (++x > 0);`	true	false

Está estrictamente prohibido ejecutar la tabla antes de registrar por escrito la predicción personal.

Comprender con exactitud cómo el compilador y la Máquina Virtual de Java (JVM) evalúan los operadores y administran el estado de las variables en memoria es el pilar fundamental sobre el cual se construyen los condicionales y la toma de decisiones complejas.

3. Sección 2: Decisiones con `if-else` y Operador Ternario

Las estructuras condicionales permiten bifucar el flujo de un programa en función de la veracidad de una o más condiciones booleanas. A continuación, resolverá tres retos contextualizados en escenarios reales del desarrollo de software.

Reto 1: Clasificación de Edades para Acceso a Plataforma Educativa

Diseñe una lógica condicional en cascada (`if-else if-else`) que categorice a los usuarios de una plataforma de aprendizaje en línea según su edad, determinando los permisos de perfil y la retención de datos asociada.

- **Requisitos de Entrada:** Una variable de tipo entero denominada `edadUsuario` (ej. `10` , `15` , `28` , `68`).
- **Condiciones Lógicas:**
  - Si `edadUsuario` es menor a `12` , clasificar como "**Infante**" (requiere tutor de cuenta).
  - Si `edadUsuario` está entre `12` y `17` (inclusive), clasificar como "**Adolescente**" (acceso restringido a foros comunitarios).
  - Si `edadUsuario` está entre `18` y `64` (inclusive), clasificar como "**Adulto**" (acceso a funcionalidades estándar).
  - Si `edadUsuario` es mayor o igual a `65` , clasificar como "**Adulto Mayor**" (acceso con interfaz adaptada).
- **Salida Esperada:** Impresión en consola indicando la categoría asignada y el perfil de acceso activado.

Reto 2: Sistema de Aprobación de Notas y Asistencia

Desarrolle un evaluador de rendimiento académico para validar si un estudiante aprueba un curso de programación considerando tanto el puntaje obtenido como su porcentaje de asistencia obligatoria.

- **Requisitos de Entrada:** Una variable de tipo decimal `calificacionFinal` (de `0.0` a `100.0`), una variable de tipo entero `porcentajeAsistencia` (de `0` a `100`), y una variable booleana `entregoProyectoFinal` .
- **Condiciones Lógicas:**
  - Un estudiante **Aaprueba** el curso si su `calificacionFinal` es mayor o igual a `70.0` **Y** (`&&`) su `porcentajeAsistencia` es mayor o igual al `80%` .
  - Excepción de recuperación: Si no cumple la asistencia o la nota mínima, pero `entregoProyectoFinal` es verdadero (`true`) **Y** la `calificacionFinal` es al menos `60.0` , el estado será "**Pendiente de Examen Extraordinario**".
  - Si no cumple ninguna de las anteriores, el estado será "**Reprobado**".
- **Salida Esperada:** Un mensaje claro por consola especificando el dictamen académico del alumno.

Reto 3: Cálculo de Tarifas de Entrada y Descuentos

Cree un programa que determine el precio de venta de una entrada para un evento tecnológico en función del tipo de boleto adquirido.

- **Requisitos de Entrada:** Un valor de tipo `String` denominado `tipoEntrada` ("VIP", "ESTUDIANTE", "GENERAL", "ADULTO_MAYOR") y un valor base de entrada `precioBase = 100.0` .
- **Condiciones Lógicas:**

    - Boleto `"VIP"`: Recibe un 10% de recargo sobre el precio base por beneficios exclusivos.
    - Boleto `"ESTUDIANTE"`: Recibe un 50% de descuento sobre el precio base.
    - Boleto `"ADULTO_MAYOR"`: Recibe un 40% de descuento sobre el precio base.
    - Boleto `"GENERAL"`: Se cobra la tarifa base sin modificaciones.
- **Salida Esperada:** Imprimir en consola el desglose del tipo de entrada y el monto final a pagar.

Subsección de Simplificación: Refactorización a Operador Ternario

El **operador ternario** (`condición ? expresión_si_verdadero : expresión_si_falso`) permite evaluar condicionales binarios simples de manera concisa en una sola línea de código.

Examine el siguiente código tradicional:

```
String estadoConexion;
if (cookieSesionActiva == true) {
    estadoConexion = "ACTIVA";
} else {
    estadoConexion = "INACTIVA";
}
```

- **Consigna de Refactorización:** Escriba en su clase de práctica una única línea de código equivalente a la estructura anterior, sustituyendo todo el bloque `if-else` por una asignación directa utilizando el operador ternario.

Dominar las evaluaciones binarias mediante `if-else` y operadores ternarios proporciona una flexibilidad enorme; sin embargo, cuando nos enfrentamos a múltiples alternativas discretas sobre un mismo elemento, Java ofrece la estructura de selección múltiple `switch`.

4. Sección 3: Selección Múltiple con `switch` (Tradicional y Moderno)

Caso Práctico: Gestor de Días de la Semana y Tipo de Jornada

En las aplicaciones empresariales es habitual transformar valores numéricos o códigos discretos en días hábiles o tipos de jornada operativa. Se solicita implementar un programa interactivo que procese el número de un día de la semana (1 para Lunes, 7 para Domingo) y determine el tipo de jornada laboral correspondiente.

Requisitos de Implementación:

El estudiante debe escribir en la clase `Main` dos versiones independientes que resuelvan la misma lógica de negocio:

1. **Implementación 1 (Sintaxis Tradicional):** Uso de la instrucción `switch` clásica con etiquetas `case`, sentencias de interrupción explícitas `break` y un caso `default` para manejar entradas inválidas.
2. **Implementación 2 (Sintaxis Moderna):** Uso de la expresión `switch` introducida en versiones recientes de Java, empleando la notación de flecha (`->`).

Explicación Pedagógica de la Sintaxis Moderna: La notación de flecha (`->`) elimina estructuralmente el riesgo de caída en cascada (*fall-through*) involuntaria al no requerir la palabra clave `break`. Además, permite retornar un valor directamente a una variable, convirtiendo el `switch` de una sentencia de control (*statement*) a una expresión evaluable (*expression*).

Tabla Comparativa de Sintaxis `switch`

Criterio de Comparación	Sintaxis Tradicional (`switch`)	Sintaxis Moderna (`switch` con `->`)
Mecanismo de Control de Flujo	Requiere la instrucción `break` al final de cada caso; de lo contrario, el flujo cae en cascada ejecutando los casos subsiguientes.	No requiere instrucción `break`. Solamente se ejecuta la expresión o bloque directamente asociado a la derecha de la flecha.
Concisión y Retorno de Valores	Funciona principalmente como bloque de sentencias. Asignar un resultado exige declarar una variable previa y mutarla en cada `case`.	Puede actuar como expresión que retorna un valor directamente, permitiendo inicializar variables de forma inmutable (`final`) y limpia.

Integrar operadores aritméticos, evaluaciones condicionales compuestas con `if-else` y selecciones múltiples con `switch` es el paso natural para la construcción de sistemas empresariales robustos.

5. Sección 4: Reto Integrador: Simulador Práctico de Caja e Inventario

Escenario Empresarial: Sistema de Facturación y Liquidación en Tienda Tecnológica

Usted ha sido designado para construir el motor principal de liquidación e inventario para un establecimiento comercial de tecnología y licencias de software. El sistema debe calcular el costo total de una compra, aplicar descuentos condicionales por tipo de cliente, procesar cupones de descuento adicionales y calcular los impuestos de ley (IVA).

Reglas de Negocio del Sistema:

1. **Cálculo del Subtotal Base:**
   - Cada unidad o paquete de licencias tiene un costo unitario estándar de **$250.00**.

- El subtotal base se calcula mediante la multiplicación aritmética: `cantidad * 250.00`.

2. **Descuento por Categoría de Cliente (vía `**switch**`):**
   - Cliente **"VIP"**: Recibe un **20%** de descuento sobre el subtotal base.
   - Cliente **"REGULAR"**: Recibe un **10%** de descuento sobre el subtotal base.
   - Cliente **"NUEVO"**: Recibe un **5%** de descuento sobre el subtotal base.
   - Cualquier otra categoría no recibe descuento comercial (**0%**).
3. **Validación de Cupón Promocional y Monto Mínimo (vía `**if-else**` y operadores lógicos):**
   - Si el cliente presenta el código de cupón `"PROMO1 año"` **Y** el subtotal tras el primer descuento es mayor o igual a **500.00**, se otorga un descuento fijo adicional de **50.00**.
   - Si no cumple ambas condiciones simultáneamente, el descuento por cupón es **$0.00**.
4. **Cálculo de Impuestos y Monto Final:**
   - Sobre la base imponible resultante (subtotal menos todos los descuentos aplicados), se calcula el **16% de IVA** (`monto * 0.16`).
   - El total final a cobrar es la suma de la base imponible más el IVA calculado.

Casos de Prueba (*Test Cases*) con Resultados Esperados:

El estudiante debe configurar los siguientes datos de entrada directamente en su código para validar la exactitud técnica de su programa:

Caso de Prueba A:

- **Datos de Entrada:** `cantidad = 3`, `tipoCliente = "VIP"`, `cupon = "PROMO1 año"`.
- **Desglose Pasos de Cálculo:**
  - Subtotal Base: 3 \times $250.00 = $750.00
  - Descuento VIP (20%): -$150.00 \rightarrow \text{Subtotal con descuento} = $600.00
  - Validación de Cupón: Es `"PROMO1 año"` y $600.00 \ge $500.00 \rightarrow \text{Aplica} -$50.00
  - Base Imponible: $600.00 - $50.00 = $550.00
  - IVA (16% de $550.00): +$88.00
- **Resultado Esperado Final a Imprimir: Total a pagar = $638.00**

Caso de Prueba B:

- **Datos de Entrada:** `cantidad = 1`, `tipoCliente = "NUEVO"`, `cupon = "PROMO1 año"`.
- **Desglose Pasos de Cálculo:**
  - Subtotal Base: 1 \times $250.00 = $250.00
  - Descuento NUEVO (5%): -$12.50 \rightarrow \text{Subtotal con descuento} = $237.50
  - Validación de Cupón: Es `"PROMO1 año"` pero $237.50 < $500.00 \rightarrow \text{NO aplica} ($0.00)
  - Base Imponible: $237.50

    - IVA (16% de $237.50): +$38.00
- **Resultado Esperado Final a Imprimir: Total a pagar = $275.50**

Incluso cuando la lógica algorítmica parece correcta en la fase de diseño, durante la escritura de código en Java pueden introducirse fallas sintácticas o de tipo. Saber diagnosticar y corregir estos inconvenientes requiere dominar el depurador (*Debugger*) del entorno de desarrollo.

6. Sección 5: Análisis y Depuración de Código (Debugging en IntelliJ IDEA)

A continuación, se presentan cuatro fragmentos independientes de código Java que contienen errores clásicos en el desarrollo de software. Copie cada fragmento en clases independientes dentro de su paquete `com.taller.practico`, utilice el depurador de IntelliJ IDEA para inspeccionar las variables y aplique las correcciones requeridas.

Fragmento 1: Falla en Condición Lógica (Asignación vs Comparación)

```
package com.taller.practico;

public class ErrorAsignacion {
    public static void main(String[] args) {
        boolean cookieActiva = false;

        // PUNTO DE INSPECCIÓN: Analice el operador utilizado dentro del parentesis del 'if'
        if (cookieActiva = true) {
            System.out.println("Estado de sesión: Sesión del usuario activa.");
        } else {
            System.out.println("Estado de sesión: Sesión inactiva.");
        }
    }
}
```

Fragmento 2: Comparación Incorrecta de Cadenas de Texto (`String`)

```
package com.taller.practico;

public class ErrorComparacionString {
    public static void main(String[] args) {
        // Generación explícita de dos objetos String en posiciones distintas de memoria
        String proveedor1 = new String("OneTrust");
```

```
        String proveedor2 = "OneTrust";

        // PUNTO DE INSPECCIÓN: Evalúe por qué '==' compara referencias de memoria y no contenido
        if (proveedor1 == proveedor2) {
            System.out.println("Los proveedores coinciden exactamente.");
        } else {
            System.out.println("Los proveedores son distintos según la comparación por referencia.");
        }
    }
}
```

Fragmento 3: Caída en Cascada Involuntaria (*Fall-Through*) en `switch` Tradicional

```
package com.taller.practico;

public class ErrorFallThrough {
    public static void main(String[] args) {
        int opcionCategoria = 1;

        // PUNTO DE INSPECCIÓN: Observe la ejecución secuencial al omitir sentencias de parada
        switch (opcionCategoria) {
            case 1:
                System.out.println("Categoría Selección: Lunes (Jornada Laboral)");
            case 2:
                System.out.println("Categoría Selección: Martes (Jornada Laboral)");
            case 3:
                System.out.println("Categoría Selección: Miércoles (Jornada Laboral)");
            default:
                System.out.println("Día no identificado");
        }
    }
}
```

Fragmento 4: Pérdida de Precisión por División Entera

```java
package com.taller.practico;

public class ErrorPrecision {
    public static void main(String[] args) {
        int tareasCompletadas = 3;
        int tareasTotales = 4;

        // PUNTO DE INSPECCIÓN: Verifique por qué (3 / 4) trunca los decimales a 0 antes de multiplicar por 100
        // PISTA DE SOLUCIÓN: Utilice un moldeo de tipo '(double)' o cambie un literal a flotante '4.0'
        double porcentajeAvance = (tareasCompletadas / tareasTotales) * 100;

        System.out.println("El porcentaje de avance es: " + porcentajeAvance + "%");
    }
}
```

Guía Paso a Paso para la Depuración (*Debugging*) en IntelliJ IDEA:

1. **Colocación de Breakpoints (Puntos de Interrupción):**
   - En el editor de código, haga clic sobre el margen izquierdo (a la derecha del número de línea) en la primera línea dentro del método `main`. Aparecerá un círculo rojo que marca la pausa de la ejecución.
2. **Ejecución en Modo Debug:**
   - Haga clic derecho sobre la clase con el error y seleccione la opción **Debug 'NombreDeLaClase.main()'** (o presione `Shift + F9`). La ejecución se detendrá en la línea marcada en resaltado amarillo.
3. **Avance Paso a Paso (*Step Over*):**
   - Presione la tecla `**F8**` (**Step Over**) para ejecutar la instrucción actual y avanzar a la siguiente línea.
   - Observe la pestaña **Variables** en el panel inferior del IDE para visualizar cómo cambian los valores en tiempo real dentro de la memoria.
4. **Inspección de Expresiones (*Evaluate Expression*):**
   - Presione `Alt + F8` sobre cualquier variable o expresión condicional para abrir la calculadora de expresiones y verificar qué valor devuelve exactamente la JVM.

Lista de Verificación Final (*Checklist* de Autoevaluación)

Valide de manera autónoma el cumplimiento de cada uno de los objetivos de este taller práctico:

- [ ] El proyecto `TallerUnidad1_2` está creado en IntelliJ IDEA dentro del paquete `com.taller.practico` y contiene la clase `Main`.
- [ ] Se completó la predicción manual de la tabla de la Sección 1 y se verificaron los resultados exactos ejecutando el código en el IDE.
- [ ] Se implementaron exitosamente los 3 retos de decisión condicional (`if-else`) de la Sección 2 (Clasificación de Edades, Aprobación de Notas y Tarifas).
- [ ] Se refactorizó la asignación de variable a una expresión compacta utilizando el operador ternario (`? :`).
- [ ] Se construyó el menú de la Sección 3 en sus dos variantes: `switch` tradicional con `break` y `switch` moderno con notación de flecha (`->`).
- [ ] El Reto Integrador de la Sección 4 produce con precisión matemática los totales de **638.00** para el Caso A y **275.50** para el Caso B.
- [ ] Se identificaron y solucionaron las causas técnicas de los 4 fragmentos defectuosos en la Sección 5 empleando las herramientas del *Debugger* (`F8` y *Breakpoints*).